

Assignment 2: HyperLogLog

Hamza Ibrahim

Juan Diego

2025-10-08

1 Introduction

This paper presents an implementation and analysis of the HyperLogLog algorithm. By varying the number of registers (m), we examine how the algorithm's cardinality estimates behave and how closely they cluster around the true number of distinct elements. The HyperLogLog algorithm is a probabilistic method for estimating the number of unique values in large data streams. It achieves this efficiently by using a fixed number of registers, resulting in low and predictable memory usage. Finally, a series of experiments are conducted to analyze how the number of registers affects the algorithm's accuracy, evaluating changes in estimation error as m increases or decreases.

2 Implementation

The implementation of the HyperLogLog algorithm can be summarized as follows:

- **Hashing:** Each input value is hashed using a fixed array of 32 random hexadecimal numbers. The hash function applies bitwise operations to generate a uniformly distributed 32-bit output.
- **Index function (f):** Determines which register a value will update. The algorithm uses a fixed-size array of registers, and f maps each hashed value to one of these registers.
- **Rho function:** Computes the number of leading zeros plus one in the 32-bit hashed value. This rho value is used to probabilistically estimate the cardinality.

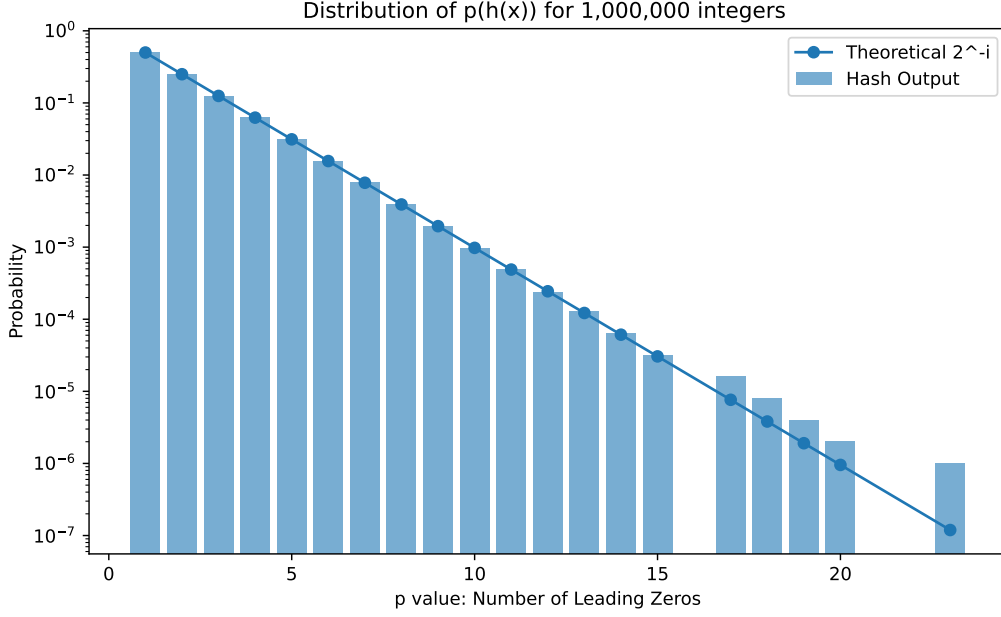


Figure 1: The y-axis (in log scale) represents the probability that a given number will have a certain number of leading zeros. As the number of leading zero increases, we observe a clear downward trend where the probability decreases confirming that $Pr[p(y) = i] = 2^{-i}$ consistent with the theoretical 2^{-i} .

For each hashed value, its rho value is stored in the corresponding register only if it is larger than the current register value. After all values are processed, the algorithm estimates the number of distinct elements using a probabilistic formula. Intuitively, if a register contains a large number of leading zeros, it indicates that the hash function has produced a rare pattern, which suggests a higher number of unique elements in the dataset. Aggregating these observations across all registers gives an accurate estimate of cardinality while using minimal memory.

In order to verify the correctness of our algorithm, we performed JUnits Tests. We were given input files and their expected answers for each of the main methods. Therefore, all we had to do was perform tests that read each input line, apply the selected method, and then use assert equals to check that the test result matches the expected result from the answers file. For the hashing method, we also created a plot based on 1,000,000 values on which we can see that about 50 percent of the values only have one leading zero, which follows the algorithm theory.

3 Experiments

After verifying the correctness of the HyperLogLog algorithm, the subprocess module in python was used to execute the java program. The experiment feeds into the algorithm a stream of one million distinct random integers consisting of values between 0 to 2^{32} (inclusive).

The stream is generated in python by creating a new list of one million distinct integers for each execution, which is then converted to a string and passed into the program. This process is repeated 100 times for each value of m (number of registers) which the HyperLogLog algorithm takes as a parameter in its constructor. Hence, for each m , the corresponding HyperLogLog instance outputs a cardinality estimation for 1 million distinct integers and this experiment is repeated 100 times. Since the number of distinct elements is fixed at one million, repeating the experiment multiple times allows us to compute the mean estimate and standard error, providing insight into how close the HyperLogLog estimates are to the true value.

For the experiment, 3 different values of m were chosen: 256, 1024 and 4096. These values are spaced far enough apart to see their impact.

Some preliminary experiments were ran to test the methods available to hyperloglog. For instance, the Rho method was run for a range of elements from one to one million to evaluate the hash function. The purpose of this experiment was to ensure that the distribution of the hash values produced by *Rho* satisfy $Pr[p(y) = i] = 2^{-i}$ where $p(y)$ is the number of leading zeros. This behavior can clearly be seen in Figure 1.

Additionally, an experiment was run to test the cardinality estimation for the Hyperloglog algorithm where $m = 1024$. For numbers between one million and two million (exclusive) the Hyperloglog cardinality estimation consistently returned 994627 corresponding to a relative error of 0.54%.

Table 1: The results of running Hyperloglog with different m on a stream of one million distinct integers (n). Each experiment was repeated 100 times for each m , showing the mean estimation, relative standard error, $\sigma = 1.04/\sqrt{m}$ and the number of estimates within $1 \pm \sigma$ and $2 \pm \sigma$ of the true cardinality.

m	n	σ	Mean Estimation	Std.Error	Within 1σ	Within 2σ
256	1000000	0.06500	1001235.67	0.05915	73/100	99/100
1024	1000000	0.03250	1001238.35	0.03092	66/100	98/100
4096	1000000	0.01625	1002865.66	0.01560	69/100	97/100

Table 1 highlights the results gathered from the experiment. The mean estimates for the Hyperloglog algorithm are not far from the correct estimate. It is noticeable that the relative standard error decreases as m grows. In addition, the relative standard error is observed to be very close to σ as expected from the algorithm [1]. For a better observation, we use the data to create histograms that show how the estimation errors cluster around the true cardinality, this is discussed further in the analysis section.

The following hardware and software were used to run the experiments:

- macOS Monterey: Version 12.7.6
- Processor: 2,7 GHz Dual-Core Intel Core i5 - 8GB RAM
- Software: openjdk version 17.0.14 - Python 3.12.4 - Gradle 7.4.2

4 Analysis

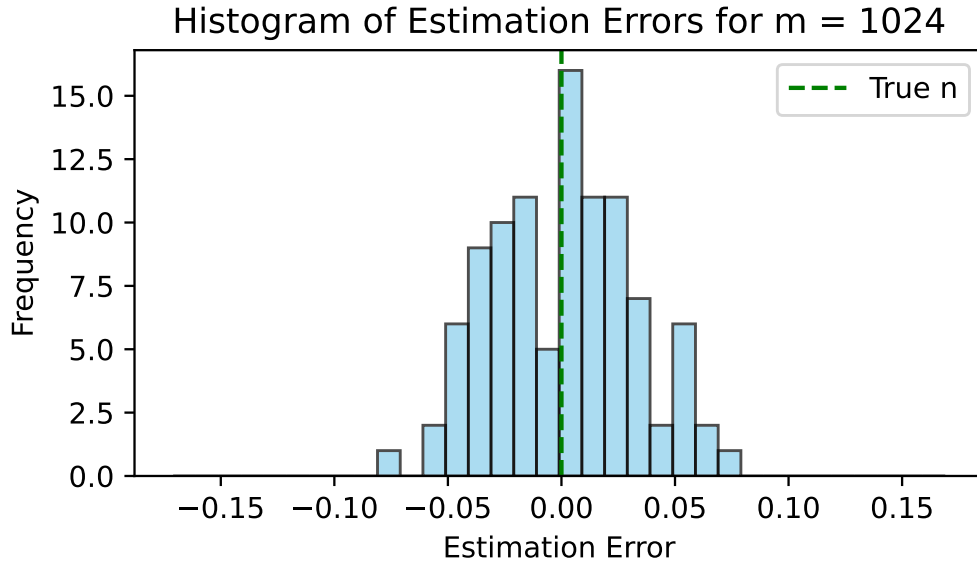


Figure 2: For $m = 1024$, Hyperloglog estimates the cardinality of 100 lists that contain one million distinct integers. Error is defined as $(estimate - n)/n$, where zero (green line) indicates a perfect estimate. The tight distribution demonstrates HyperLogLog’s accuracy.

Figure 2 shows the distribution of the estimation errors and visually highlight the results of Table 1. 98% of the estimations are within $2 \pm \sigma$ with minimum outliers. Most estimates cluster around -5% and $+5\%$.

In figure 3 we see a similar histogram to figure 2 but with a slightly wider distribution where estimates cluster around -10% and $+10\%$. This wider spread is consistent with the larger standard error reported in Table 1, for $m = 256$ ($\sigma = 6.50\%$) compared to $m = 1024$ ($\sigma = 3.25\%$). The increase in standard error is directly related to the number of registers. As m increases, the estimation variance decreases resulting in a tighter cluster around the true cardinality. This trend is further confirmed in Figure 4, where $m = 4096$ yields the lowest relative standard error among all tested configurations. Visually, the histograms for $m = 256$, $m = 1024$, and $m = 4096$ each exhibit an approximately bell-shaped distribution centered near zero estimation error, reflecting the expected statistical behavior of the HyperLogLog estimator.

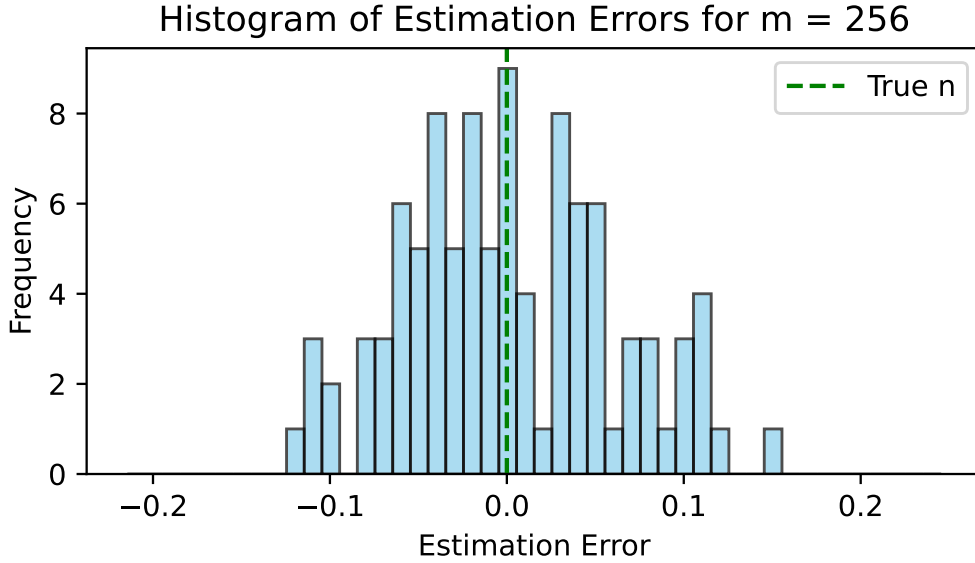


Figure 3: For $m = 256$, Hyperloglog estimation errors, similar explanation to Figure 2.

The results are somewhat limited, as the experiment only tested streams of one million distinct integers. It would have been valuable to explore how the results vary with different stream sizes or by repeating the experiment 300–400 times to obtain a better statistical sample. Nonetheless, the findings provide meaningful insight into HyperLogLog’s accuracy and efficiency.

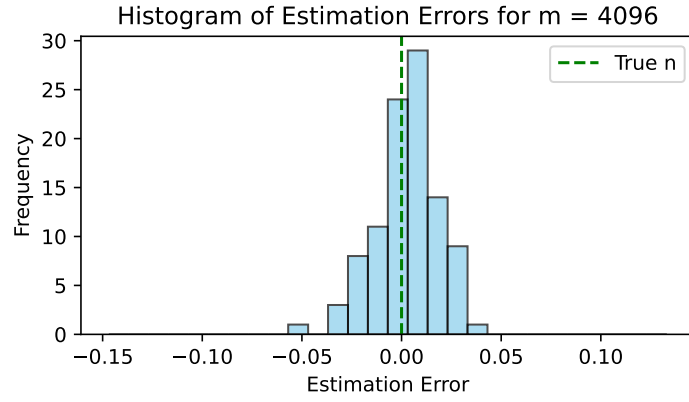


Figure 4: For $m = 4096$, Hyperloglog estimation errors, similar explanation to Figure 2.

References

- [1] Philippe Flajolet et al. “Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm”. In: *Discrete mathematics & theoretical computer science* Proceedings (2007).